Web Development

CSE343
Section 4

Advanced JavaScript

ES6 Features – Shortcuts for Optimizing Code

- ES6 (short for ECMAScript 6) is a major update to the JavaScript language, released in 2015.
- Before ES6:
  - Code was verbose (too many lines for simple tasks).
  - There were no proper ways to write classes, modules, or constants.
  - Asynchronous code (like waiting for network data) was hard to manage.

- ES6 introduced many new features that made JavaScript cleaner, more consistent, and developer-friendly. Introduced modern syntax like:
  - let/const (We saw in previous lab)
  - arrow functions
  - template literals
  - destructuring, (New concepts to discuss today)
  - spread/rest
  - Classes
  - Modules import and export



Arrays

Arrays

- An array is an ordered, zero-based collection of values.
- Arrays are objects in JavaScript (so they have methods and properties).
- Elements can be any type (numbers, strings, objects, functions, even other arrays).
- Arrays are dynamic: they grow or shrink automatically.
- It is a common practice to declare arrays with the const keyword.

```
const myArray = [30,40,50];
const myArray1 = ['Hello', 'Hi', "GoodMorning"];
```

Arrays (Cont.)

- Accessing an array and length behaviour:

```
const arr = [10, 20, 30];
console.log(arr[0]);     // 10
console.log(arr.length); // 3

// length is writable:
arr.length = 2; // truncates -> [10, 20]
arr.length = 5; // length becomes 5, empty slots added: [10, 20, <3 empty>]
```

Arrays (Cont.)

- Mutating methods that modify the original array

```
const m = [1,2,3];
m.push(4);      // [1,2,3,4]
m.pop();        // [1,2,3]  (returns 4)
m.shift();      // [2,3]    (removes first)
m.unshift(0);   // [0,2,3]
m.splice(1,1);  // remove 1 at index 1 -> [0,3]
m.sort();       // sorts in place (string compare by default)
m.reverse();    // reverse in place
m.fill(7, 1, 3); // fill indices [1,3) with 7
```

Arrays (Cont.)

- Mutating methods that return a new array or a new value
  - **splice**() -> The splice() method can be used to add new items to an array:
    - The first parameter (2) defines the position where new elements should be added (spliced in). The second parameter (2) defines how many elements should be removed. The rest of the parameters ("Lemon" , "Kiwi") define the new elements to be added. The splice() method **returns an array with the deleted items**.

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
const array = fruits.splice(2, 2, "Lemon", "Kiwi");
console.log(fruits);
console.log(array);
```

```
▸ (4) ['Banana', 'Orange', 'Lemon', 'Kiwi']
▸ (2) ['Apple', 'Mango']
```

Arrays (Cont.)

- Non-Mutating methods that return a new array or a new value
  - **concat()** -> The concat() method creates a new array by merging existing arrays:

```
const myGirls = ["Cecilie", "Lone"];
const myBoys = ["Emil", "Tobias", "Linus"];

const myChildren = myGirls.concat(myBoys);
console.log(myChildren);
```

```
▶ (5) ['Cecilie', 'Lone', 'Emil', 'Tobias', 'Linus']
```

Arrays (Cont.)

- Non-Mutating methods that return a new array or a new value
  - **map()** -> transform every item in an array into a new item => returns a new array of the same length. arr.map((value, index, array) => newValue)

```
const nums = [1, 2, 3];
const doubled = nums.map(n => n * 2); // [2, 4, 6]

const users = [{name: 'Mazen', age: 30}, {name: 'Anna', age: 25}];
const names = users.map(u => u.name); // ['Mazen', 'Anna']
```

Arrays (Cont.)

- Non-Mutating methods that return a new array or a new value
  - **filter()** -> keep only items that match a condition => returns a (possibly smaller) new array.
    - arr.filter((value, index, array) => boolean)

```
const nums = [1, 2, 3, 4, 5];
const evens = nums.filter(n => n % 2 === 0); // [2, 4]

const users = [
  {name: 'Mazen', active: true},
  {name: 'Sam', active: false}
];
const activeUsers = users.filter(u => u.active); // [{name: 'Mazen', active: true}]
```

Arrays (Cont.)

- **Array Destructuring** assigns elements of an array (or any iterable) to variables using bracket syntax.

```
const arr = [10, 20, 30];
const [x, y, z] = arr;
console.log(x, y, z); // 10 20 30

const arr1 = [1, 2, 3, 4];
const [, second, , fourth] = arr;
console.log(second, fourth); // 2 4
```

Arrays (Cont.)

- **Spread (...)** = unpacking things and laying them out individually. (right side of equal sign)
- **Rest (...)** = packing up the leftover things into one bag. (left side of equal sign)

```
const box = [1, 2, 3];
const shelf = [0, ...box, 4]; // spread: unpack items from box
// shelf is [0, 1, 2, 3, 4]

Math.max(...box); // spread: call Math.max(1, 2, 3)
```

```
const [first, ...rest] = [10, 20, 30];
// first = 10
// rest = [20, 30]

function greet(first, ...others) {
  // others is an array of remaining args
}
greet('Hi', 'Alice', 'Bob'); // others = ['Alice','Bob']
```

Classes

Classes

- JavaScript Classes are templates for JavaScript Objects.

```
class ClassName {
  constructor() {

  }
}
```

```
class Car {
  constructor(name, year) {
    this.name = name;
    this.year = year;
  }
}

const myCar1 = new Car("Ford", 2014);
const myCar2 = new Car("Audi", 2019);
```

Inheritance: extends & super

- Enables one class to inherit properties and methods from another.
- 'extends' creates the child class, and 'super()' calls the parent constructor.
- Example:

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { console.log(`${this.name}
makes a noise.`); }
}
class Dog extends Animal {
  constructor(name, breed) {
    super(name);
    this.breed = breed;
  }
  speak() { console.log(`${this.name}
barks!`); }
}
```

Getters and Setters

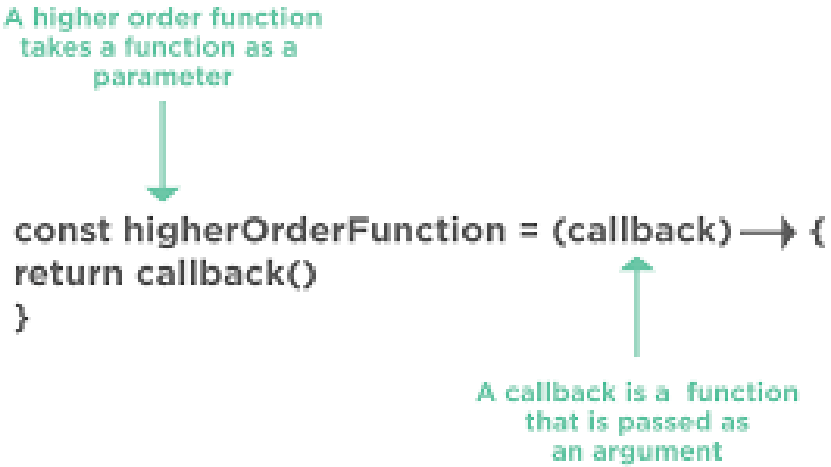
- Control access to object data with computed or validated properties.
- Example:

```
class Temperature {
  constructor(celsius) { this.celsius =
celsius; }
  get fahrenheit() { return this.celsius
* 1.8 + 32; }
  set fahrenheit(value) { this.celsius =
(value - 32) / 1.8; }
}
const temp = new Temperature(0);
console.log(temp.fahrenheit); // 32
```

Callbacks

Function Callbacks

- A callback is a function passed to another function and invoked later. It allows one function to call another when an operation completes.
- Used for events, timers, and older async APIs.
- Pitfalls: callback pyramid (callback hell), inversion of control, error handling complexity.
- Synchronous callbacks
  - The callback executes immediately after being called.
  - JavaScript waits for it to finish before moving on.
- Asynchronous callbacks
  - In asynchronous callbacks, JavaScript doesn't wait for the function to finish and continues executing other code.
  - The callback runs later after a delay or external event.
  - Common in network requests, file reading, or timers.



Example Code






```
<h2>Load User Example</h2>
<button id='loadBtn'>Load User</button>
<p id='output'></p>
<script>
  function fetchUser(sync_callback, async_callback) {
    const name = prompt('Enter your name:');
    sync_callback(name);

    const age = prompt('Enter your age:');

    setTimeout(() => {
     const user = { name: `${name}`, age: `${age}` };
     async_callback(user);
    }, 2000);

    document.getElementById('output').textContent = 'Name and Age are entered';
  }
  function greetUser(user) {
    alert(`Hello, ${user}!`);

  }

  function displayUser(user) {
    document.getElementById('output').textContent = `Name: ${user.name}, Age: ${user.age}`;

  }
  document.getElementById('loadBtn').addEventListener('click', () => fetchUser(greetUser,displayUser));
</script>
```

Sync callback. The code will run callback and finish it first to continue other codes

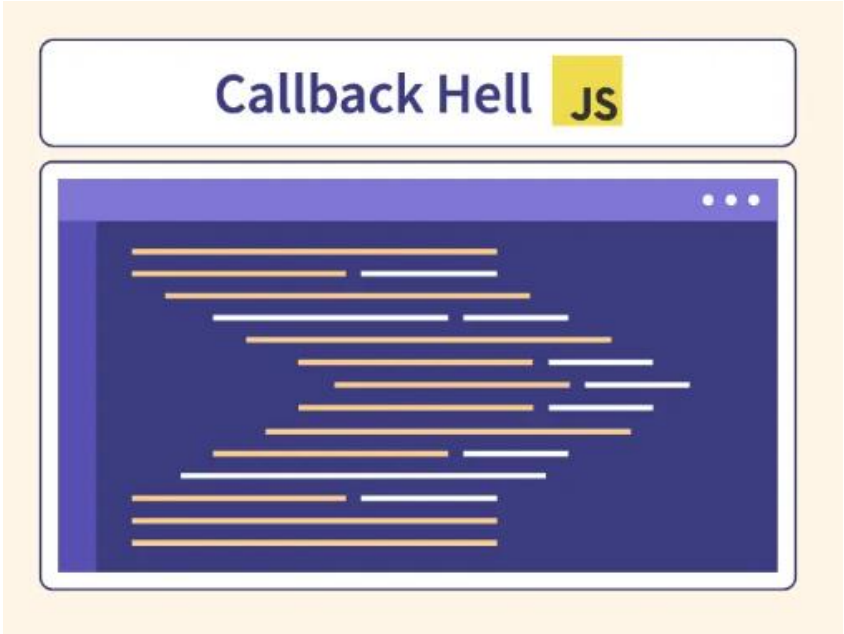
Async callback called through setTimeout. Code inside will run after 2 seconds without blocking other codes



Callback Hell

- When multiple asynchronous callbacks are nested, code becomes messy:

```
getData(() => {
  processData(() => {
    saveData(() => {
      displayData(() => {
        console.log('Done!');
      });
    });
  });
});
// Difficult to debug and maintain.
```



What is TypeScript?

Problems with Javascript

- The problem is that JavaScript allows the same function to receive numbers or strings. In the second example, + concatenates the strings instead of adding the numbers.
- JavaScript allows this reassignment. That may be perfectly valid JavaScript, but it could cause a problem if the rest of the application expects age to be a number.

```
function add(a, b) {
    return a + b;
}

console.log(add(5, 10));     // 15
console.log(add("5", "10")); // "510"
```

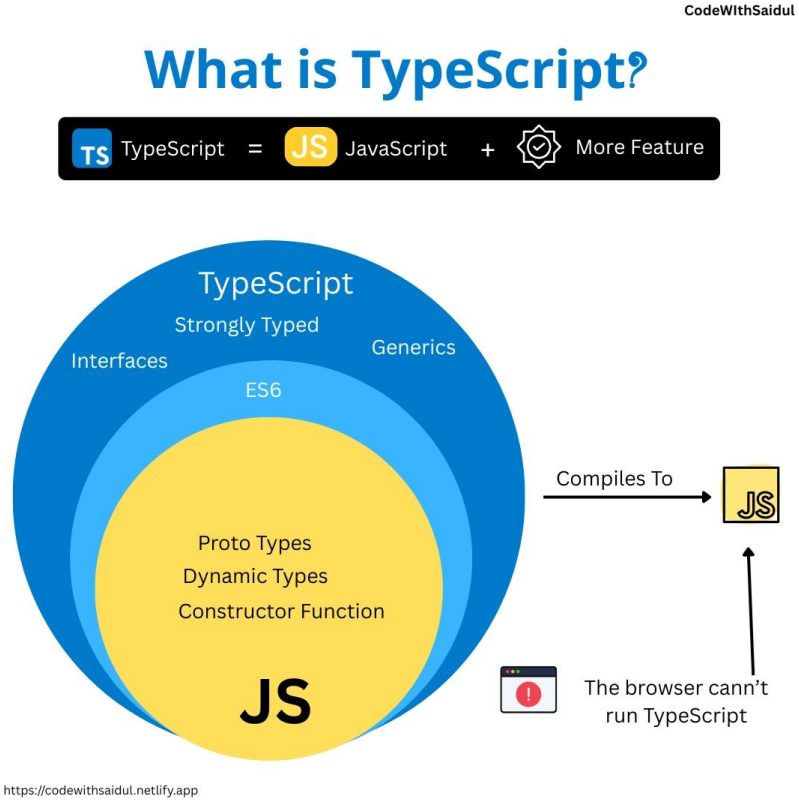
```
let age = 24;

age = "twenty-four";
```

Intro to Typescript

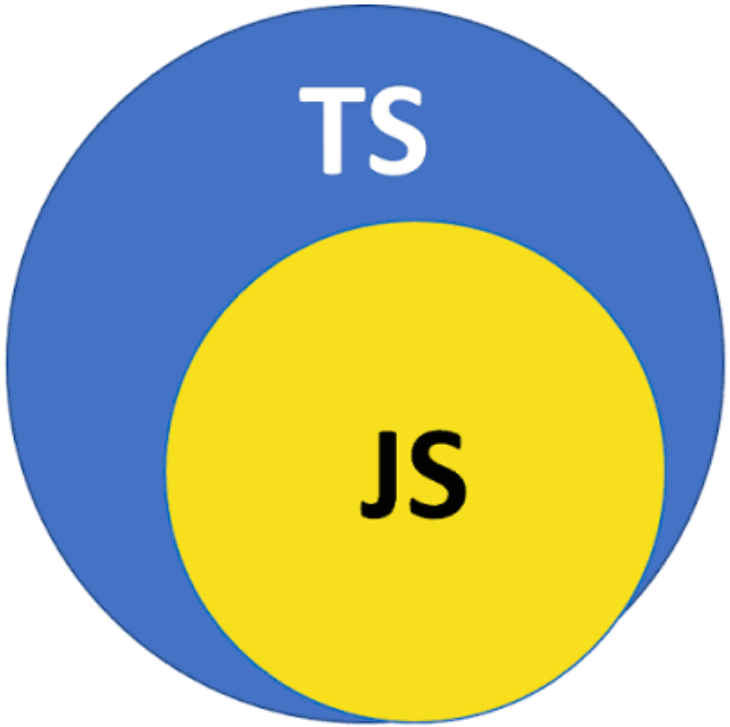
- TypeScript is a programming language built on top of JavaScript that adds a static type system.
  - JavaScript is the language that browsers execute directly.
  - TypeScript adds features such as type annotations, interfaces, and compile-time checking.
  - TypeScript is transpiled into JavaScript so that the resulting code can run in a browser or another JavaScript runtime.

TypeScript does not replace JavaScript's runtime. It helps check code before execution, but the generated JavaScript is what normally executes.



Why do we need Typescript?

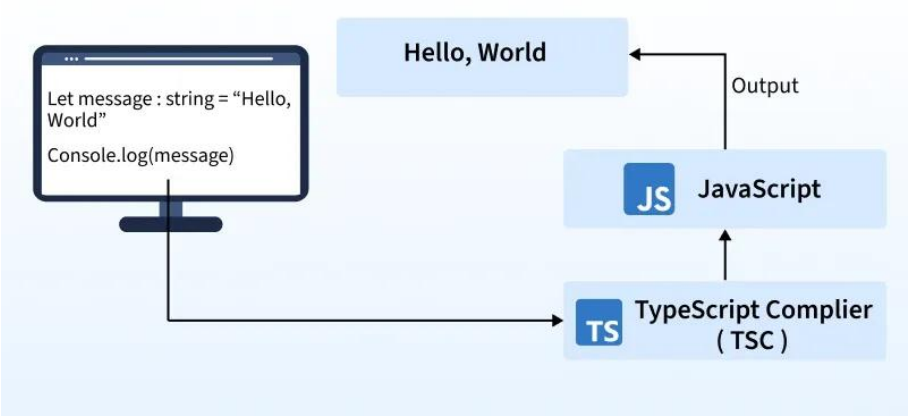
- Catch errors before running the program
- Make code easier to understand



What is Transpilation?

Transpilation

- In simple terms:
  - Compilation transforms source code into another form that can be executed or processed.
  - Transpilation transforms source code from one language or language version into another language or version at a similar level of abstraction.



Transpilation (cont.)

- TypeScript uses a compiler called tsc to check types and transform TypeScript source code into JavaScript.
- For example, create a file called app.ts:

```
let studentName: string = "Mazen";
let grade: number = 95;

console.log(`${studentName} scored ${grade}`);
```

- After transpilation, the generated JavaScript (app.js) looks like this:

```
let studentName = "Mazen";
let grade = 95;

console.log(`${studentName} scored ${grade}`);
```

Process of Transpilation

- The developer writes app.ts.
- The TypeScript compiler checks the code for type errors.
- The compiler generates JavaScript output, subject to its configuration.
- The browser or Node.js executes the generated JavaScript.

Typescript Syntax

Typescript Syntax

- Type annotations and type inference
  - An annotation explicitly specifies a variable's type.

```
let studentName: string = "Ahmed";
let grade: number = 90;
let passed: boolean = true;
```

```
let grades: number[] = [90, 85, 95];

let students: string[] = ["Ahmed", "Sara", "Mazen"];
```

The basic types are:

Type	Example
`string`	`"Hello"`
`number`	`42`, `3.14`
`boolean`	`true`, `false`

```
function multiply(a: number, b: number): number {
    return a * b;
}
```

Setting up Typescript

Typescript Project Setup

- Install Node.Js if you don’t already have it. Node.js — Download Node.js®
- Initialize the project with **npm init -y**, This creates a file named package.json. It is the configuration file that describes your Node.js project, including its name, scripts, and dependencies. The -y flag automatically accepts the default answers instead of asking questions interactively.
- **npm install --save-dev typescript**, This command installs TypeScript as a development dependency. Why a development dependency? Because we need TypeScript to check and compile our source code during development. The resulting JavaScript is what the browser normally executes.
- Initialize the TypeScript compiler configuration using **npx tsc --init**. Tsc is the typescript compiler. Npx allows you to run a package executable, including the locally installed TypeScript compiler, without needing to install it globally. The tsconfig.json file controls how TypeScript checks and compiles the project's files.
- Compile TypeScript into JavaScript using **npx tsc**

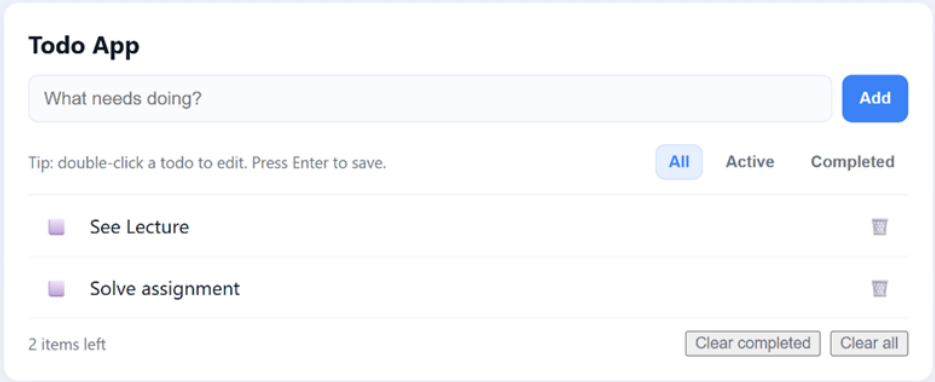
Typescript Project Setup

- In the tsconfig.json, you can modify the default code with the following code to the right if you only want the js output file without any mapping or linker related files too. Also, the code to the right works with both typescript and generated javascript files found in the root path of the project. If you want to split them into different folders, some slight modification is needed (try it yourself ☺)

- Once the js file is provided by the typescript compiler, it can be used as any javascript file to be used as the src of the script in the html file.

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ES2020",
    "strict": true,
    "noEmitOnError": true
  },
  "include": ["*.ts"]
}
```

Lab Task


Todo App
What needs doing?
Add
Tip: double-click a todo to edit. Press Enter to save.
All
Active
Completed
See Lecture
Solve assignment
2 items left
Clear completed
Clear all